
HR Employee Attrition Analysis

Identifying the drivers of employee turnover and predicting at-risk employees with machine learning & Power BI

Author	Rahul
Domain	People Analytics / HR
Dataset	IBM HR Analytics Employee Attrition (1,470 employees, 35 features)
Tech stack	Python · Pandas · NumPy · Scikit-learn · Matplotlib · Seaborn · Power BI
Deliverables	EDA · Predictive model · 3-page interactive dashboard

Project report — prepared for academic / portfolio submission.

1. Executive Summary

Employee attrition is expensive: replacing a single employee can cost between 50% and 200% of their annual salary once recruitment, lost productivity and ramp-up time are accounted for. This project analyses the IBM HR Analytics dataset of 1,470 employees to answer two questions: **what drives people to leave**, and **which current employees are most at risk**. The work combines an exploratory analysis and a predictive model in Python with an interactive three-page Power BI dashboard that scores every employee with an attrition-risk band.

The headline finding is that **overtime is the single strongest driver of attrition** — employees working overtime leave at roughly three times the rate of those who do not (30.5% vs 10.4%). Risk is further concentrated among Sales Representatives, single employees, the lowest income bands and the first two years of tenure. A balanced Logistic Regression model identifies about 62% of actual leavers, giving HR an early-warning list for proactive intervention.

1,470
Employees analysed

16.1%
Overall attrition

3x
Overtime risk multiple

0.80
Best model ROC-AUC

2. Problem Statement & Business Context

When an employee leaves, the organisation loses institutional knowledge and incurs the cost of sourcing, hiring and training a replacement. HR teams typically react *after* resignations land on their desk — at which point retention is rarely possible. The objective of this project is to shift HR from a reactive to a **proactive** posture by:

- **Diagnosing the drivers** — quantifying which factors (compensation, role, workload, demographics, tenure) are most strongly associated with leaving.
- **Predicting individual risk** — assigning every current employee an attrition probability so managers can target retention conversations before it is too late.
- **Operationalising the insight** — delivering the results in a Power BI dashboard that a non-technical HR partner can filter and act on.

3. Dataset Overview

The analysis uses the publicly available **IBM HR Analytics Employee Attrition & Performance** dataset (Kaggle). It contains one row per employee with 35 attributes spanning demographics, compensation, job role, satisfaction scores and tenure.

Property	Value
Records	1,470 employees
Features	35 columns (numeric + categorical)
Target variable	Attrition (Yes / No)
Class balance	237 leavers vs 1,233 stayers — 16.1% positive class (imbalanced)
Missing values	None
Example features	Age, MonthlyIncome, OverTime, JobRole, Department, MaritalStatus, YearsAtCompany, JobSatisfaction, DistanceFromHome, etc.

The 16/84 class imbalance is the central modelling challenge. A naive model could score 84% accuracy simply by predicting that *everyone stays* — while being useless to HR. Every modelling decision below (class

weighting, choice of recall as the key metric) flows from this fact.

4. Methodology & Workflow

The project follows a standard end-to-end analytics pipeline:



5. Data Cleaning & Preparation

Three columns carry no information because they hold the same value for every employee, and the employee ID is just an identifier — all four are dropped. The target is mapped to a 1/0 integer so it can be modelled, and a defensive assertion confirms there are no missing values.

```
# Drop zero-variance / non-predictive columns
df = df.drop(columns=["EmployeeCount", "StandardHours", "Over18", "EmployeeNumber"])

# This dataset has no nulls, but always verify rather than assume
assert df.isnull().sum().sum() == 0, "Missing values found!"

# Encode the target: Yes -> 1 (left), No -> 0 (stayed)
df["Attrition"] = df["Attrition"].map({"Yes": 1, "No": 0})
```

Why this matters: zero-variance features add noise and dimensionality without predictive value, and a model cannot train on the raw text labels 'Yes'/'No'. The `assert` is a small habit that catches data-quality regressions early rather than letting them silently corrupt results.

6. Exploratory Data Analysis

Before modelling, EDA surfaces the relationships in the data. Attrition rate is computed across each categorical feature, leaver-vs-stayer distributions are compared for the key numeric features, and a correlation heatmap checks for redundancy among numeric columns.

6.1 Attrition rate across categorical features

```
cat_features = ["Department", "JobRole", "OverTime", "MaritalStatus",  
               "BusinessTravel", "Gender", "EducationField"]  
for col in cat_features:  
    rate = df.groupby(col)["Attrition"].mean().sort_values(ascending=False)  
    sns.barplot(x=rate.values, y=rate.index, palette="rocket") # mean of 0/1 = rate
```

Grouping by a category and taking the **mean of the 0/1 target** is a compact trick to get the attrition *rate* per group. The charts below make the drivers visually obvious.

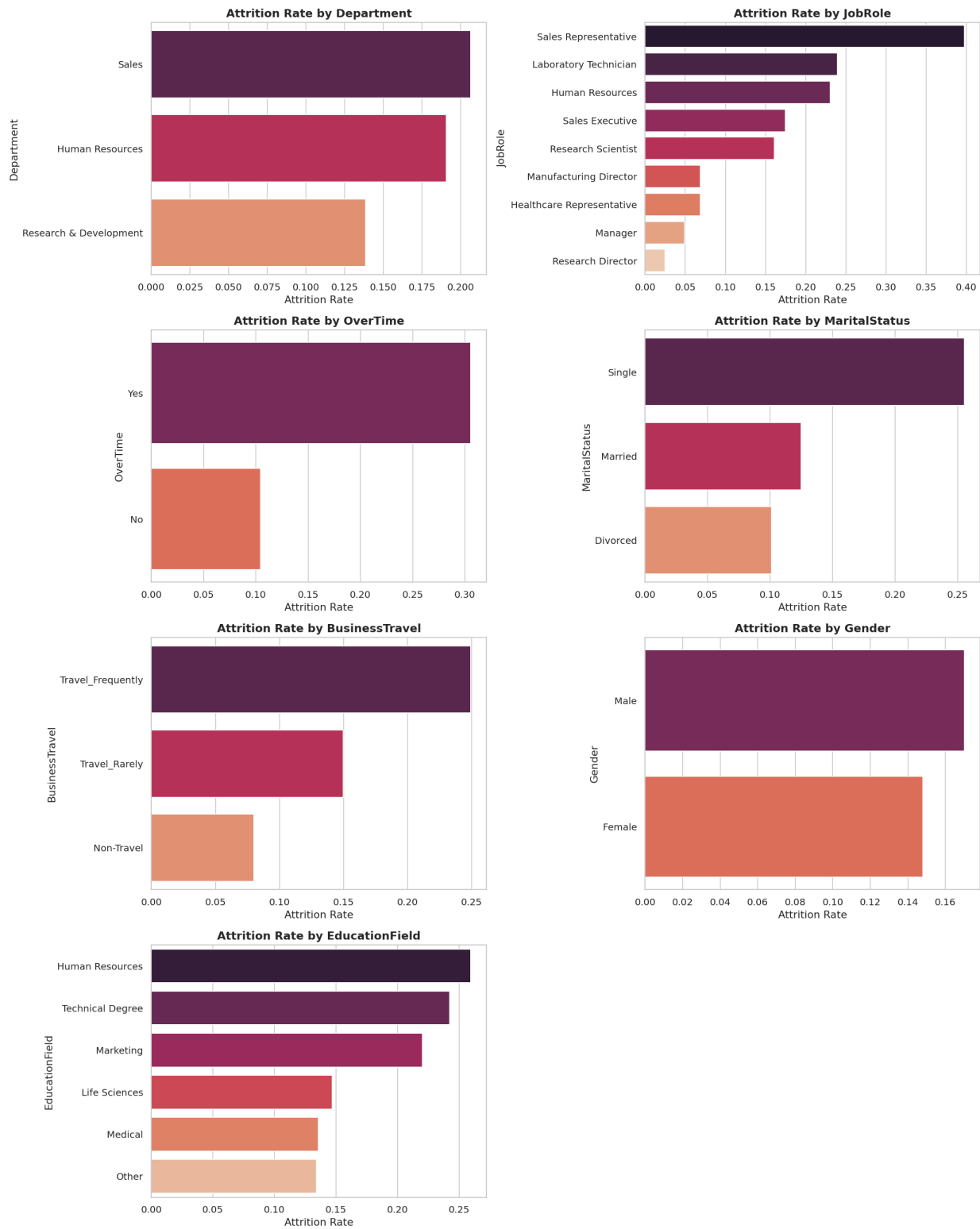


Figure 1 — Attrition rate by each categorical feature. Overtime, Sales Representative role, and Single marital status stand out sharply.

6.2 Numeric distributions — leavers vs stayers

Kernel-density plots overlay the distribution of leavers (orange) and stayers (teal) for each numeric feature. Where the two curves separate, the feature carries signal. `common_norm=False` normalises each group independently so the much smaller leaver group is still visible.

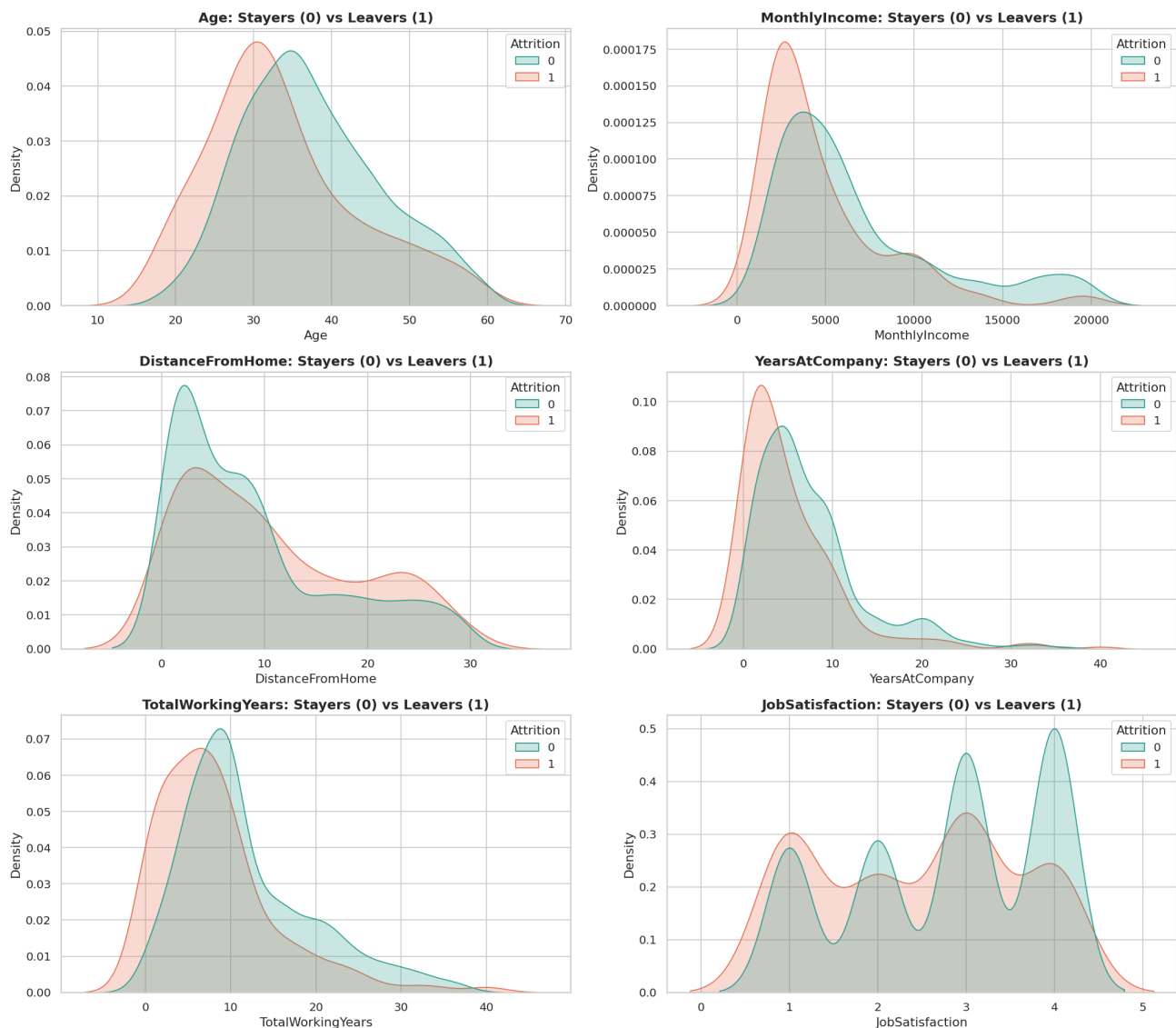


Figure 2 — Leavers skew younger, lower-income, shorter-tenured and live farther from work; satisfaction is lower among leavers.

6.3 Correlation heatmap

The heatmap checks for multicollinearity among numeric features. Expected clusters appear — the various tenure columns (`YearsAtCompany`, `YearsInCurrentRole`, `YearsWithCurrManager`) move together, as do `JobLevel`, `MonthlyIncome` and `TotalWorkingYears`. This is useful context for interpreting feature importance later.

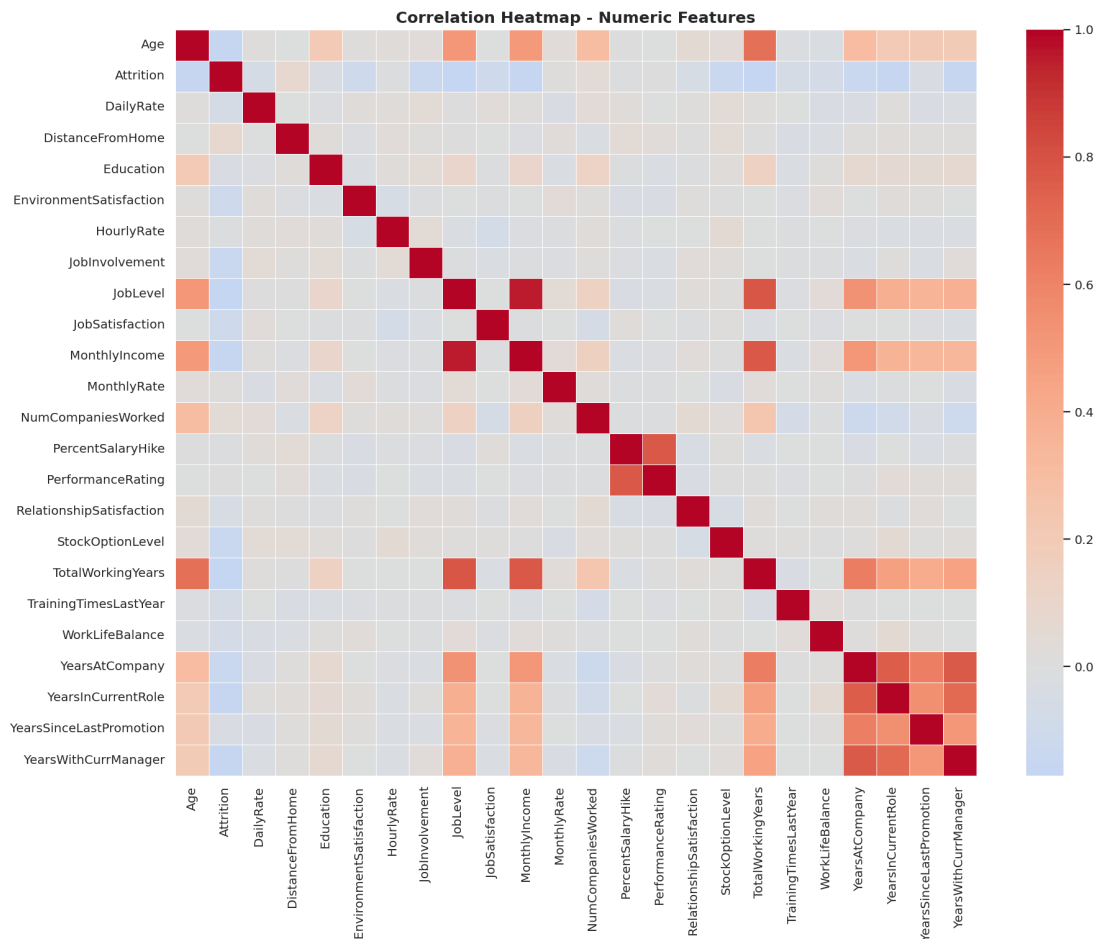


Figure 3 — Correlation heatmap of numeric features.

Key EDA insights

Driver	Finding
OverTime	30.5% attrition with overtime vs 10.4% without — strongest single signal
Job role	Sales Representatives leave most (~40%), then Lab Technicians (~24%)
Marital status	Single employees leave at 25.5% vs 12.5% married / 10.1% divorced
Age / tenure	Leavers average 33.6 yrs vs 37.6; risk peaks in the first 1–2 years
Income	Attrition concentrates in the lowest monthly-income bands

7. Feature Engineering & Encoding

Machine-learning models need numeric input. Binary text columns (e.g. OverTime, Gender) are label-encoded to 0/1, while multi-class columns (e.g. JobRole, Department) are one-hot encoded so no false ordinal relationship is implied. `drop_first=True` avoids the dummy-variable trap.

```
binary_cols = [c for c in df.select_dtypes("object").columns if df[c].nunique() == 2]
multi_cols = [c for c in df.select_dtypes("object").columns if df[c].nunique() > 2]

le = LabelEncoder()
for col in binary_cols:
    # OverTime, Gender, ...
    df_model[col] = le.fit_transform(df_model[col])

df_model = pd.get_dummies(df_model, columns=multi_cols, drop_first=True) # JobRole, ...

X = df_model.drop(columns="Attrition")
y = df_model["Attrition"]

# Stratified split preserves the 16/84 class ratio in BOTH train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

scaler = StandardScaler() # needed for Logistic Regression
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

Two details that matter: `stratify=y` guarantees the rare leaver class is proportionally represented in the test set (without it, a random split could leave too few leavers to evaluate fairly). The scaler is fit on *train only* and then applied to test — fitting on the full data would leak test information into training.

8. Predictive Modelling

Two models are trained, both with `class_weight="balanced"` so the algorithm penalises mistakes on the rare leaver class more heavily — the standard way to stop an imbalanced model from ignoring the minority. A Logistic Regression provides an interpretable baseline; a Random Forest captures non-linear interactions.

```
# Baseline - Logistic Regression
log_reg = LogisticRegression(max_iter=2000, class_weight="balanced", random_state=42)
log_reg.fit(X_train_sc, y_train)

# Random Forest - 300 trees, depth-capped to limit overfitting
rf = RandomForestClassifier(n_estimators=300, max_depth=10,
                           class_weight="balanced", random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
```

Model	Accuracy	Leaver Recall	Leaver Precision	Leaver F1	ROC-AUC
Logistic Regression (baseline)	75%	62%	35%	44%	0.798
Random Forest	84%	11%	50%	18%	0.779

Interpreting the results honestly. For an HR early-warning system, **recall on the leaver class is the metric that matters** — missing an employee who is about to quit (a false negative) is far costlier than flagging a loyal employee for a check-in (a false positive). On that metric the Logistic Regression is the better tool: it catches roughly **62% of actual leavers**, whereas the Random Forest at its default 0.5 threshold is conservative and catches very few. The Random Forest posts a higher headline accuracy, but that is largely the accuracy of predicting the majority ‘stay’ class — the exact trap the class imbalance sets. The two ROC-AUC scores (~0.78–0.80) are similar, confirming both models rank risk reasonably; the difference is where the decision threshold sits. A natural next step is to tune the Random Forest threshold to raise its recall.

8.1 Confusion matrix & ROC curves

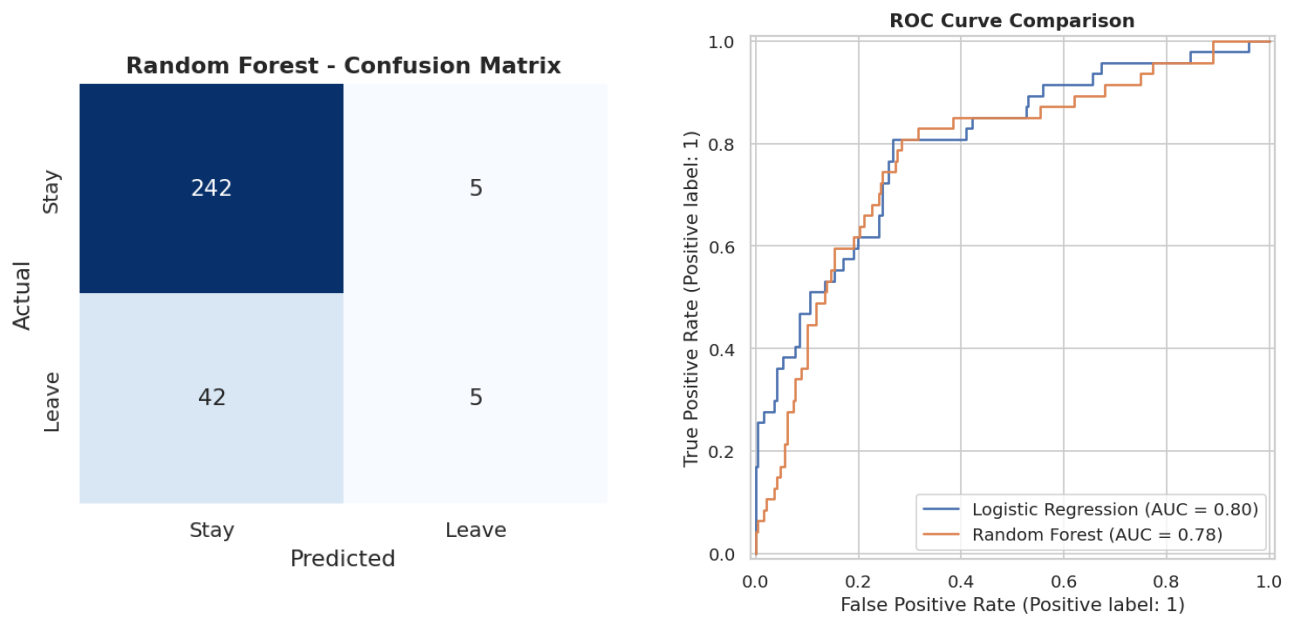


Figure 4 — Random Forest confusion matrix (left) and ROC curves for both models (right). The ROC curves sit well above the diagonal chance line.

9. What Actually Drives Attrition (Feature Importance)

The Random Forest exposes a feature-importance ranking — how much each variable contributes to its predictions. This is the ‘why’ behind the model and the bridge back to business action.

```
importances = (pd.Series(rf.feature_importances_, index=X.columns)
                .sort_values(ascending=False).head(15))
sns.barplot(x=importances.values, y=importances.index, palette="viridis")
```

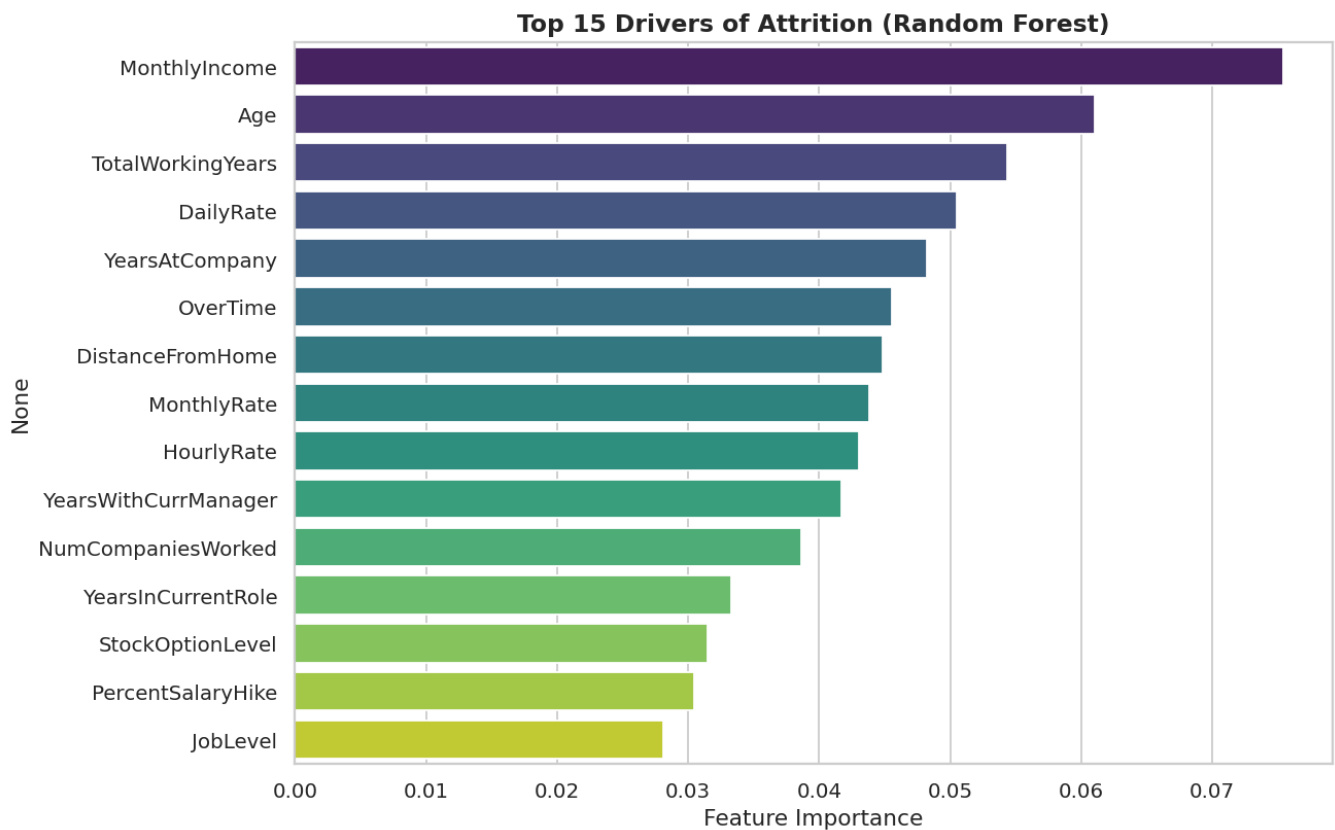


Figure 5 — Top 15 attrition drivers ranked by Random Forest importance.

Compensation and experience-related features (MonthlyIncome, Age, TotalWorkingYears, DailyRate, YearsAtCompany) dominate the top of the ranking, with **OverTime** the highest single behavioural lever. This aligns with the EDA: pay, life-stage/tenure, and workload are the levers HR can realistically influence.

10. Power BI Dashboard

The Python model scores every employee and exports an enriched file (hr_attrition_powerbi.csv) carrying an AttritionRiskScore and a Low / Medium / High RiskBand. That feeds a three-page Power BI dashboard designed for a non-technical HR audience.

```
# Score every employee and bucket into risk bands for the dashboard
df_export["AttritionRiskScore"] = rf.predict_proba(X)[: , 1].round(3)
df_export["RiskBand"] = pd.cut(df_export["AttritionRiskScore"],
                               bins=[0, 0.3, 0.6, 1.0], labels=["Low", "Medium", "High"])
df_export.to_csv("hr_attrition_powerbi.csv", index=False)
```

Core DAX measures used on top of the imported table:

```
Attrition Rate = DIVIDE( CALCULATE(COUNTROWS(HR), HR[Attrition]=1), COUNTROWS(HR) )
High Risk Count = CALCULATE( COUNTROWS(HR), HR[RiskBand]="High" )
```

Page 1 — Overview

KPI cards (headcount, attrition rate, average tenure, average income) sit above attrition broken down by department, tenure bucket and marital status, plus an overall attrition split.

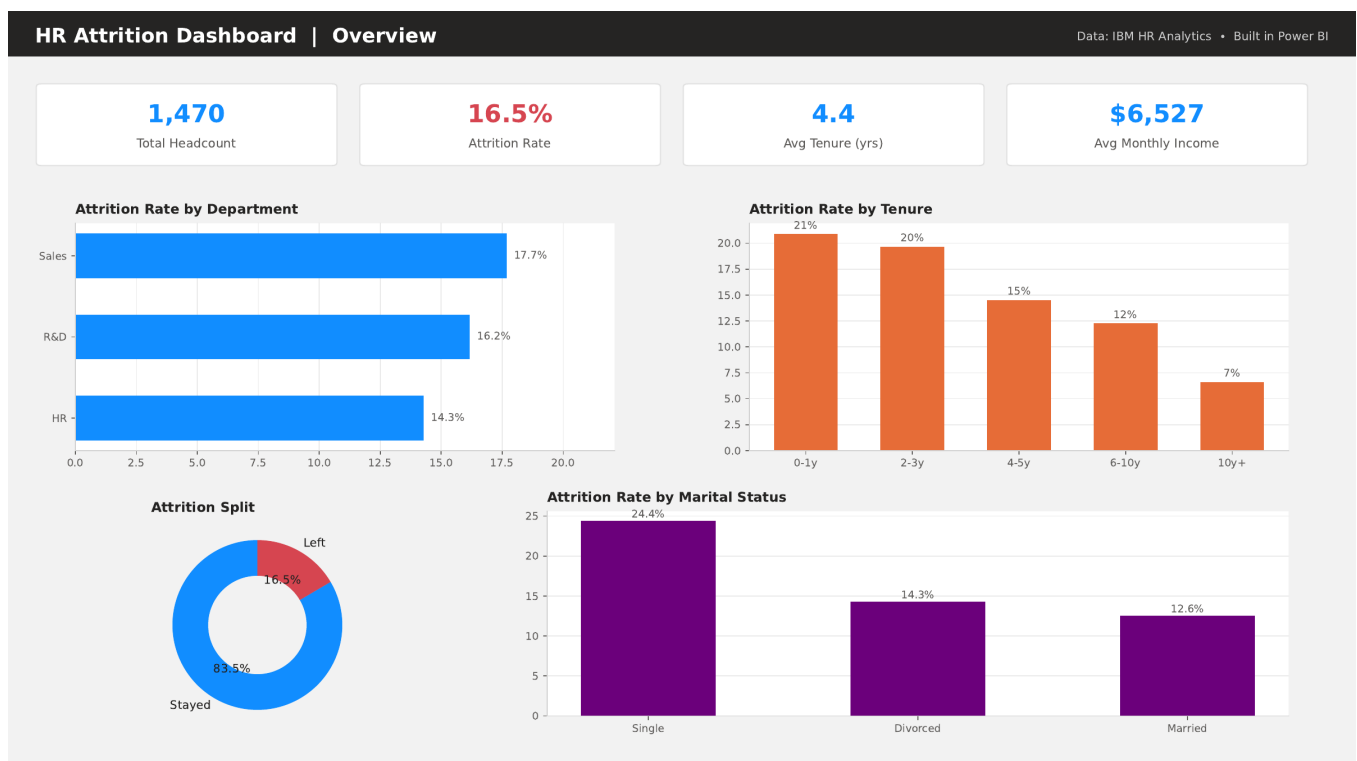


Figure 6 — Dashboard Page 1: Overview KPIs and headline breakdowns.

Page 2 — Attrition Drivers

Focuses on the 'why': the overtime gap, role-level attrition, income bands and the age-group trend line that together explain most of the variation.

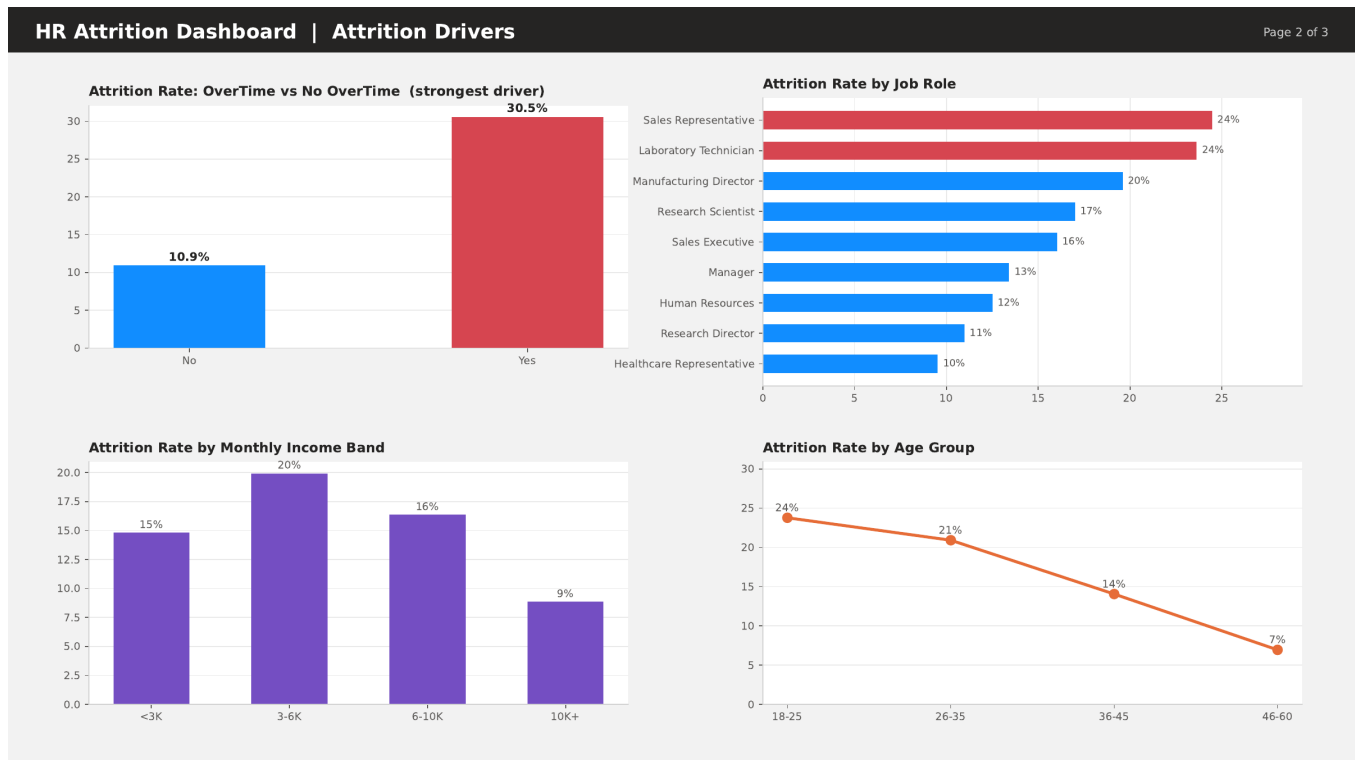


Figure 7 — Dashboard Page 2: the drivers behind attrition.

Page 3 — High-Risk Watchlist

The actionable page: risk-band counts, high-risk employees by department, and a ranked watchlist table with conditional formatting on the risk score, ready for managers to act on in their next 1:1s.

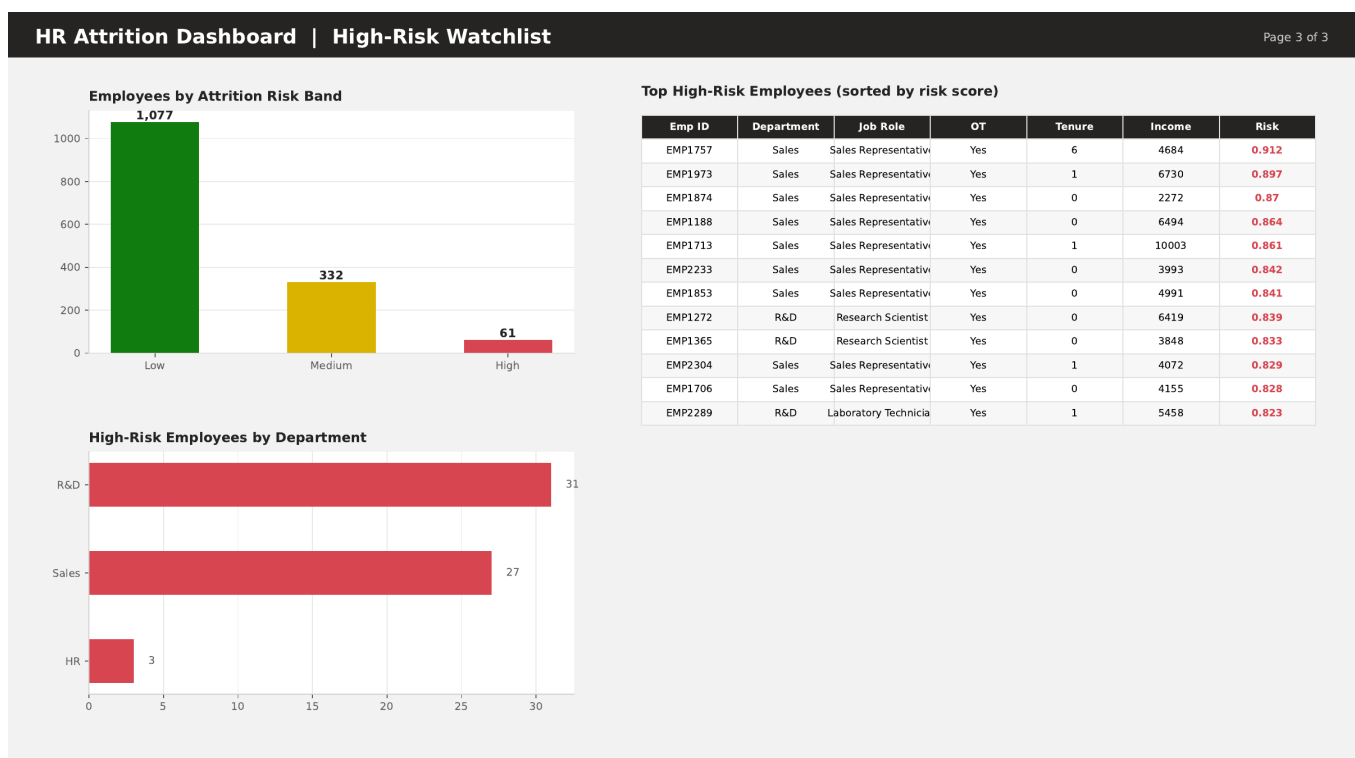


Figure 8 — Dashboard Page 3: the High-Risk Watchlist.

11. Consolidated Key Insights

- **Overtime is the dominant driver.** Attrition is ~3× higher among employees working overtime (30.5% vs 10.4%) and OverTime is the top behavioural feature in the model.
- **Risk is role-concentrated.** Sales Representatives (~40%) and Laboratory Technicians (~24%) leave far more than other roles, pointing to specific job-design problems.
- **Early tenure is fragile.** The first one-to-two years carry the highest attrition; the risk then falls steadily with each additional year of service.
- **Pay matters at the bottom.** The lowest monthly-income bands show the highest attrition, and MonthlyIncome is the single most important feature in the model.
- **Life stage shows up.** Single employees leave at roughly double the rate of married ones, and leavers are on average four years younger than stayers.

12. Business Recommendations

Insight	Recommended action
Overtime drives exits	Cap or fairly compensate sustained overtime; rebalance workloads, especially in Sales and Lab Technicians
High-risk roles	Run targeted retention and job-design reviews for Sales Representatives and Lab Technicians (career path)
Early-tenure churn	Strengthen onboarding and schedule structured 6 / 12 / 18-month check-ins for new joiners.
Low-pay attrition	Benchmark the lowest salary bands against market and adjust where the gap is driving exits.
Proactive watchlist	Feed the model's High-risk list into managers' 1:1s so retention conversations happen before resignation.

13. Conclusion, Limitations & Future Work

This project delivers an end-to-end view of employee attrition — from raw data to a deployable Power BI watchlist. The analysis consistently points to overtime, low pay, specific high-churn roles and early tenure as the factors HR should act on first, and the predictive model turns those patterns into a per-employee risk score for proactive retention.

Limitations

- The dataset is a static IBM sample and may not reflect any specific company's workforce; findings should be re-validated on real organisational data.
- At the default 0.5 threshold the Random Forest under-flags leavers; threshold tuning or resampling (SMOTE) would likely improve its recall.
- Feature importance shows association, not causation — the recommendations are hypotheses to test, not proven cause-and-effect.

Future work

- Tune the decision threshold and try gradient-boosted models (XGBoost / LightGBM) with cross-validation.
- Add SHAP values for transparent, per-employee explanations of each risk score.
- Operationalise as a scheduled refresh so the watchlist updates automatically each month.

Appendix — Reproducing the Analysis

```
pip install pandas numpy matplotlib seaborn scikit-learn
# place the IBM dataset CSV in the project folder
python hr_attrition_analysis.py
# Outputs: EDA + model PNG charts, and hr_attrition_powerbi.csv for Power BI
```

HR Employee Attrition Analysis — project report by Rahul. Python (Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn) & Power BI.